

Notebook

February 8, 2026

1 DSA and ECDSA

We've covered how DLP can be applied to encryption. DH and ECDH allow one to create a symmetric key using public-key cryptography. Unfortunately, they can't be used to create a signature. However, finite fields modulo prime and elliptic curve groups can still be used to create signatures. The most widespread algorithms for this are DSA and its elliptic curve variant ECDSA.

1.1 Definitions

1.1.1 DSA

DSA requires a hash function $Hash(x)$, a multiplicative group Z_p^* modulo a prime number p and a generator g of order q , where q is the largest prime divisor of $p - 1$ (so g is generator of the largest subgroup of prime order of group Z_p^*). We've already covered this, but to find one you need to pick a random element $z \neq 1, p - 1$ and calculate $g = z^{\frac{p-1}{q}} \bmod p$. Obviously, all these parameters are usually standardized.

Now that we have your $Hash(x)$, Z_p^* , g , q we can create a private and public key. The pair is generated this way:

1. Pick a random $x \in (0, q)$ - this is the private key
2. Compute $y = g^x \bmod p$ - this is the public key

Now we can sign a message m :

1. Pick a random nonce (number used once) $k \in (0, q)$
2. Compute $r = (g^k \bmod p) \bmod q$. If $r = 0$ go to 1.
3. Compute $s = \frac{Hash(m) + x \cdot r}{k} \bmod q$. If $s = 0$ go to 1.
4. Output (r, s) . This is the signature

To verify the signature we must check, that $r, s \in (0, q)$ and the following equality:

$$(g^{\frac{Hash(m)}{s}} \cdot y^{\frac{r}{s}} \bmod p) \bmod q = r$$

Let's check correctness:

$$(g^{\frac{Hash(m)}{s}} \cdot y^{\frac{r}{s}} \bmod p) \bmod q = (g^{\frac{Hash(m)}{s}} \cdot g^{x \cdot \frac{r}{s}} \bmod p) \bmod q$$

$$(g^{\frac{Hash(m)}{s}} \cdot g^{x \cdot \frac{r}{s}} \bmod p) \bmod q = (g^{\frac{Hash(m)}{s} + x \cdot \frac{r}{s}} \bmod p) \bmod q$$

$$(g^{\frac{Hash(m)}{s} + x \cdot \frac{r}{s}} \bmod p) \bmod q = (g^{\frac{Hash(m) + x \cdot r}{s}} \bmod p) \bmod q$$

$$(g^{\frac{Hash(m) + x \cdot r}{s}} \bmod p) \bmod q = (g^{\frac{\frac{Hash(m) + x \cdot r}{s}}{k}} \bmod p) \bmod q = (g^k \bmod p) \bmod q = r$$

1.1.2 ECDSA

In the case of ECDSA we have a curve $E(\mathbb{GF}(p))$ and some point G as generator, which has prime order q , so $q \cdot G = 0$ (Point at Infinity). We also need a hash function $Hash(x)$. To create keys we once again:

1. Pick a random $d \in (0, q)$ as our private key
2. Compute $Q = d \cdot G$ to be our public key

Signing m :

1. Pick a random nonce $k \in (0, q)$
2. Calculate $(x_1, y_1) = k \cdot G$
3. Calculate $r = x_1 \bmod q$. If $r = 0$ go to 1.
4. Calculate $s = \frac{Hash(m) + r \cdot d}{k} \bmod q$. If $s = 0$ go to 1.
5. Output (r, s) as signature.

Verification:

1. Check that $r, s \in (0, q)$
2. Compute $(x_1, y_1) = \frac{Hash(m)}{s} \cdot G + \frac{r}{s} \cdot Q$
3. Check that $r = x_1 \bmod q$

Correctness:

$$\frac{Hash(m)}{s} \cdot G + \frac{r}{s} \cdot Q = \frac{Hash(m)}{s} \cdot G + d \cdot \frac{r}{s} \cdot G = \frac{Hash(m) + d \cdot r}{s} \cdot G = k \cdot G = (x_1, y_1)$$

1.2 Problems

You might have noticed that both primitives use a random value k during signing. This is the plague of this family of algorithms, because **so much** can go wrong. DSA and ECDSA implementations can be broken by:

1. Using predictable k
2. Using the same k twice
3. Using biased k (but we'll not be covering this here)

In the first case, since we can predict k we can recover private key:

$$1. \text{ For DSA. } s = \frac{\text{Hash}(m) + x \cdot r}{k} \bmod q \implies x = \frac{s \cdot k - \text{Hash}(m)}{r} \bmod q$$

$$2. \text{ For ECDSA } s = \frac{\text{Hash}(m) + r \cdot d}{k} \bmod q \implies d = \frac{s \cdot k - \text{Hash}(m)}{r} \bmod q$$

In the second scenario, since the same k has been used to produce two different signatures:

$$k = \frac{\text{Hash}(m) + x \cdot r}{s} \bmod q \implies \frac{\text{Hash}(m_1) + x \cdot r_1}{s_1} = \frac{\text{Hash}(m_2) + x \cdot r_2}{s_2} \bmod q \implies$$

$$(\text{Hash}(m_1) + x \cdot r_1)s_2 = (\text{Hash}(m_2) + x \cdot r_2)s_1 \bmod q \implies$$

$$\text{Hash}(m_1)s_2 + x \cdot r_1 \cdot s_2 = \text{Hash}(m_2)s_1 + x \cdot r_2 \cdot s_1 \bmod q \implies$$

$$\text{Hash}(m_1)s_2 - \text{Hash}(m_2)s_1 = x(r_2 \cdot s_1 - r_1 \cdot s_2) \bmod q \implies$$

$$x = \frac{\text{Hash}(m_1)s_2 - \text{Hash}(m_2)s_1}{r_2 \cdot s_1 - r_1 \cdot s_2} \bmod q$$

So we can compute x :

$$x = \frac{\text{Hash}(m_1)s_2 - \text{Hash}(m_2)s_1}{r_2 \cdot s_1 - r_1 \cdot s_2} \bmod q$$

1.3 Task

Since the attacks are the same, we'll just be attacking DSA. There will be two sub-tasks. The server generates two key pairs. He signs the first message with the first pair using a predictable k :

```
class predictable_DSA(DSA):
    def get_k(self):
        return int(time.time())%(self.q-1)+1
```

Hint: the server may be in a different timezone

Then he generates a new k with a different function and uses them to create second and third signature (with the second key). Your goal is create a signature of message 'give' with the first key and message 'flag' with the second key, then submit them to the server. Good luck!

```
[ ]: import random
import time
```

```

from Crypto.Util.number import bytes_to_long
import hashlib
#These are the classes used on the server's side
class DSA:
    def __init__(self,g,p,q):
        self.p=p
        self.g=g
        self.q=q
        self.x=random.randint(1,q-1)
        self.y=pow(g,self.x,self.p)

    def get_k(self):
        raise NotImplementedError()
    @staticmethod
    def message_to_hashnum(m):
        if not isinstance(m,bytes):
            raise TypeError()
        return bytes_to_long(hashlib.sha256(m).digest())
    def sign(self,m):
        r=0
        s=0
        while r==0 or s==0:
            k=0
            while k==0:
                k=self.get_k()
            r=pow(g,k,self.p)%self.q
            s=((DSA.message_to_hashnum(m)+(self.x*r)%self.q)%self.
↪q)*pow(k,q-2,self.q))%self.q
            return (r,s)
    def verify(self,m,r,s,otherY):
        if r==0 or s==0: return False
        w=pow(s,self.q-2,self.q)
        u1=(DSA.message_to_hashnum(m)*w)%self.q
        u2=(r*w)%self.q
        res=(pow(self.g,u1,self.p)*pow(otherY,u2,self.p)%p)%q
        return res==r

class predictable_DSA(DSA):
    def get_k(self):
        return int(time.time())%(self.q-1)+1

class repeated_DSA(DSA):
    def get_k(self):
        global constant_k
        return constant_k

```

```

[ ]: import socket
import re
class VulnServerClient:
    def __init__(self, show=True):
        """Initialization, connecting to server"""
        self.s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
        self.s.connect(('cryptotraining.zone',1350))

    def recv_until(self, symb=b'\n>'):
        """Receive messages from server, by default till new prompt"""
        data=b''
        while True:

            data+=self.s.recv(1)
            if data[-len(symb):]==symb:
                break
        return data
    def getChallenge(self, show=True):
        data=self.recv_until()
        try:
            data=data.decode()
        except UnicodeDecodeError:
            print ('Error decoding unicode. Try connecting to server_
↪again.')
            return (None, None)
        if show:
            print (data)
        p=int(re.search(r'(?<=p=)\d+', data).group(0))
        q=(p-1)//2
        g=int(re.search(r'(?<=g=)\d+', data).group(0))
        y1=int(re.search(r'(?<=y1=)\d+', data).group(0))
        y2=int(re.search(r'(?<=y2=)\d+', data).group(0))
        mrs1=re.search(r'(?<=s1=)\(b'[a-z]+\', \d+, \d+\\)', data).
↪group(0)[1:-1].split(',')
        m1,r1,s1=mrs1[0][2:-1].encode(),int(mrs1[1]),int(mrs1[2])
        mrs2=re.search(r'(?<=s2=)\(b'[a-z]+\', \d+, \d+\\)', data).
↪group(0)[1:-1].split(',')
        m2,r2,s2=mrs2[0][2:-1].encode(),int(mrs2[1]),int(mrs2[2])
        mrs3=re.search(r'(?<=s3=)\(b'[a-z]+\', \d+, \d+\\)', data).
↪group(0)[1:-1].split(',')
        m3,r3,s3=mrs3[0][2:-1].encode(),int(mrs3[1]),int(mrs3[2])
        #print (m1.encode(),r1,s1)
        return (p,q,g,y1,y2,m1,r1,s1,m2,r2,s2,m3,r3,s3)

    def checkSolution(self,rg,sg,rf,sf, show=True):
        """Checking solution"""

```

```

        self.s.sendall((str(rg)+' '+str(sg)+' '+str(rf)+'\n'+str(sf)+'\n').encode())
        data=self.recv_until(b'\n')
        try:
            data=data.decode()
        except UnicodeDecodeError:
            print ('Error decoding unicode. Try connecting to server again.')
            return None
        if show:
            print (data)
        if data.find('flag')!=-1:
            return True
        else:
            data=self.recv_until(b'>')
            try:
                data=data.decode()
            except UnicodeDecodeError:
                print ('Error decoding unicode. Try connecting to server again.')
                return None
            if show:
                print (data)
            return False
    def __del__(self):
        self.s.close()

```

```

vs=VulnServerClient()
(p,q,g,y1,y2,m1,r1,s1,m2,r2,s2,m3,r3,s3)=vs.getChallenge()

```

```

[ ]: # You need to find x1 and x2
pDSA.x=x1
pDSA.y=y1
(r_g,s_g)=pDSA.sign(b'give')
pDSA.x=x2
pDSA.y=y2
(r_f,s_f)=pDSA.sign(b'flag')

```

```

[ ]: vs.checkSolution(r_g,s_g,r_f,s_f)

```

```

[ ]:

```

```

[ ]:

```